LAPORAN PRAKTIKUM
POSTTEST 7
ALGORITMA PEMROGAMAN LANJUT

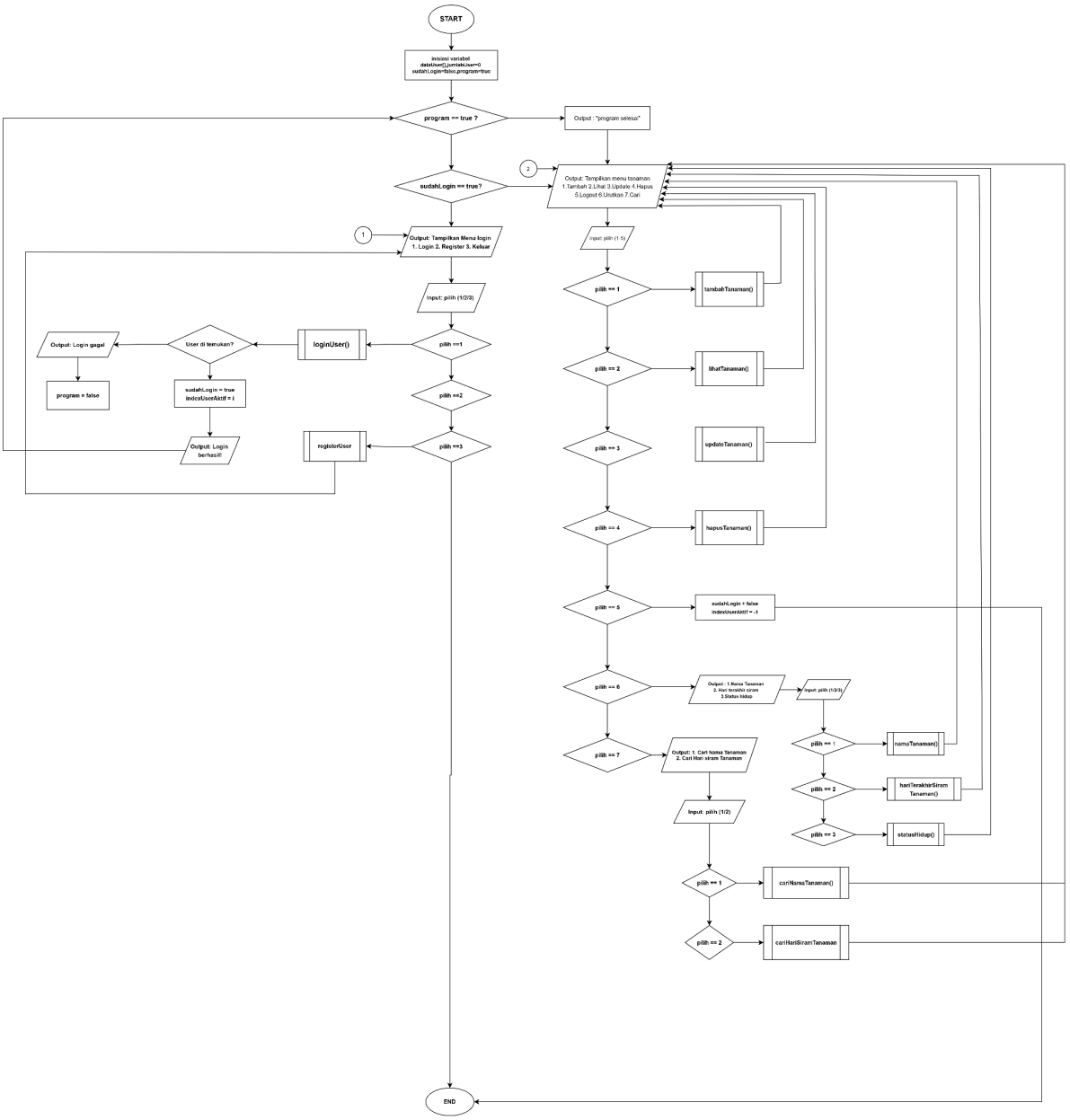
Disusun oleh:

Kikania Agarista (2509106086)

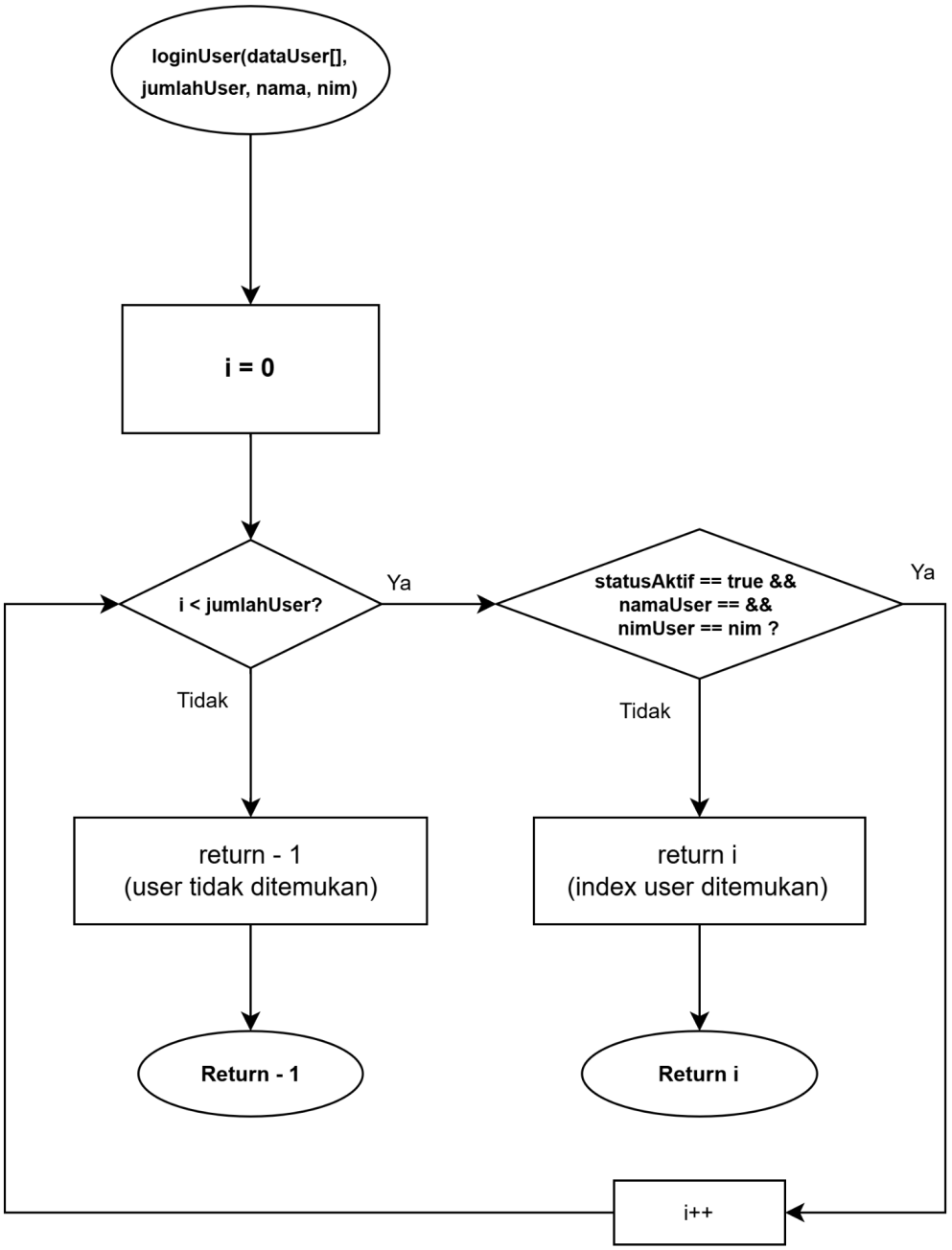
Kelas (B2'25)

PROGRAM STUDI INFORMATIKA
UNIVERSITAS MULAWARMAN
2026

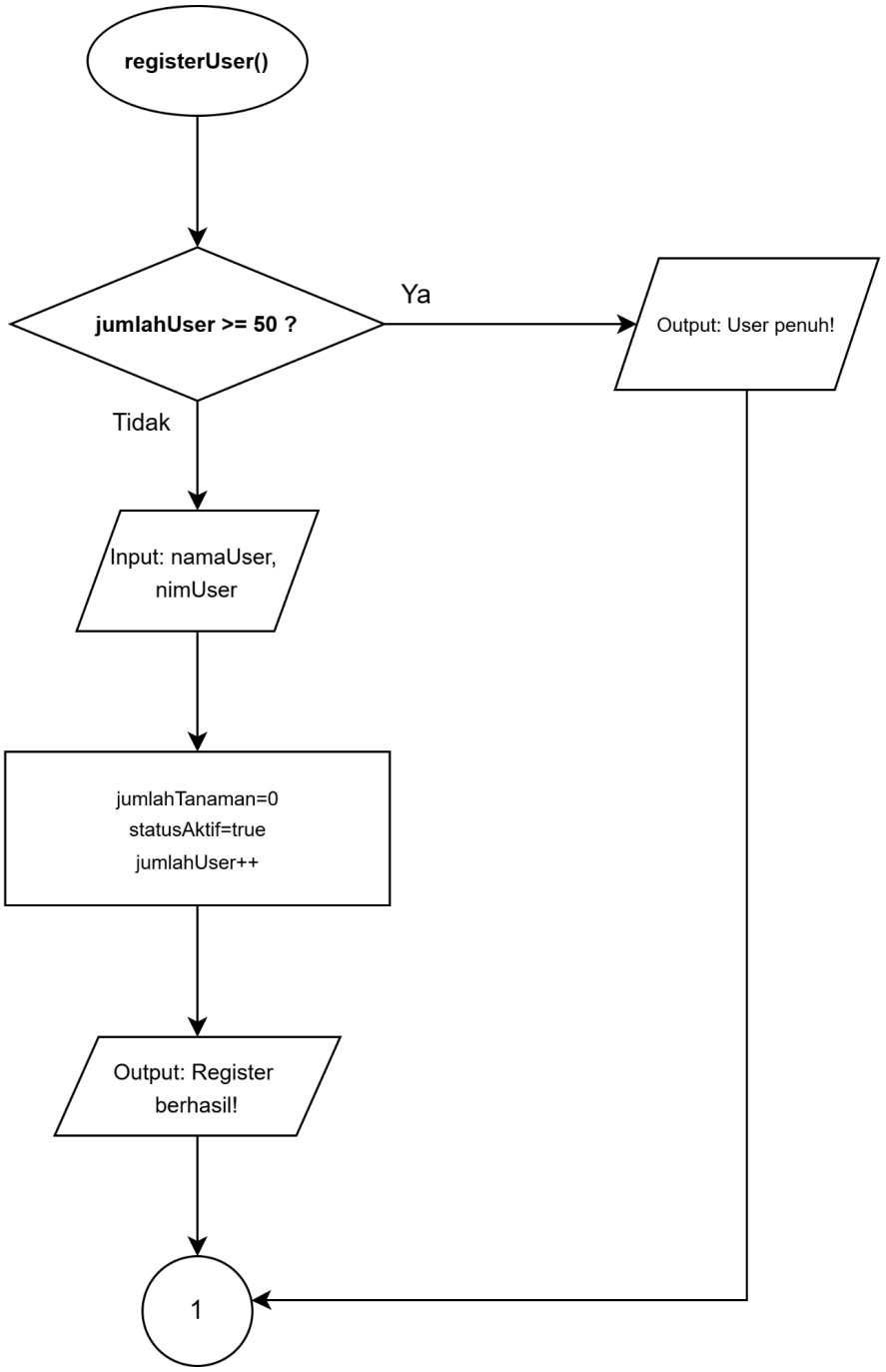
A. Flowchart



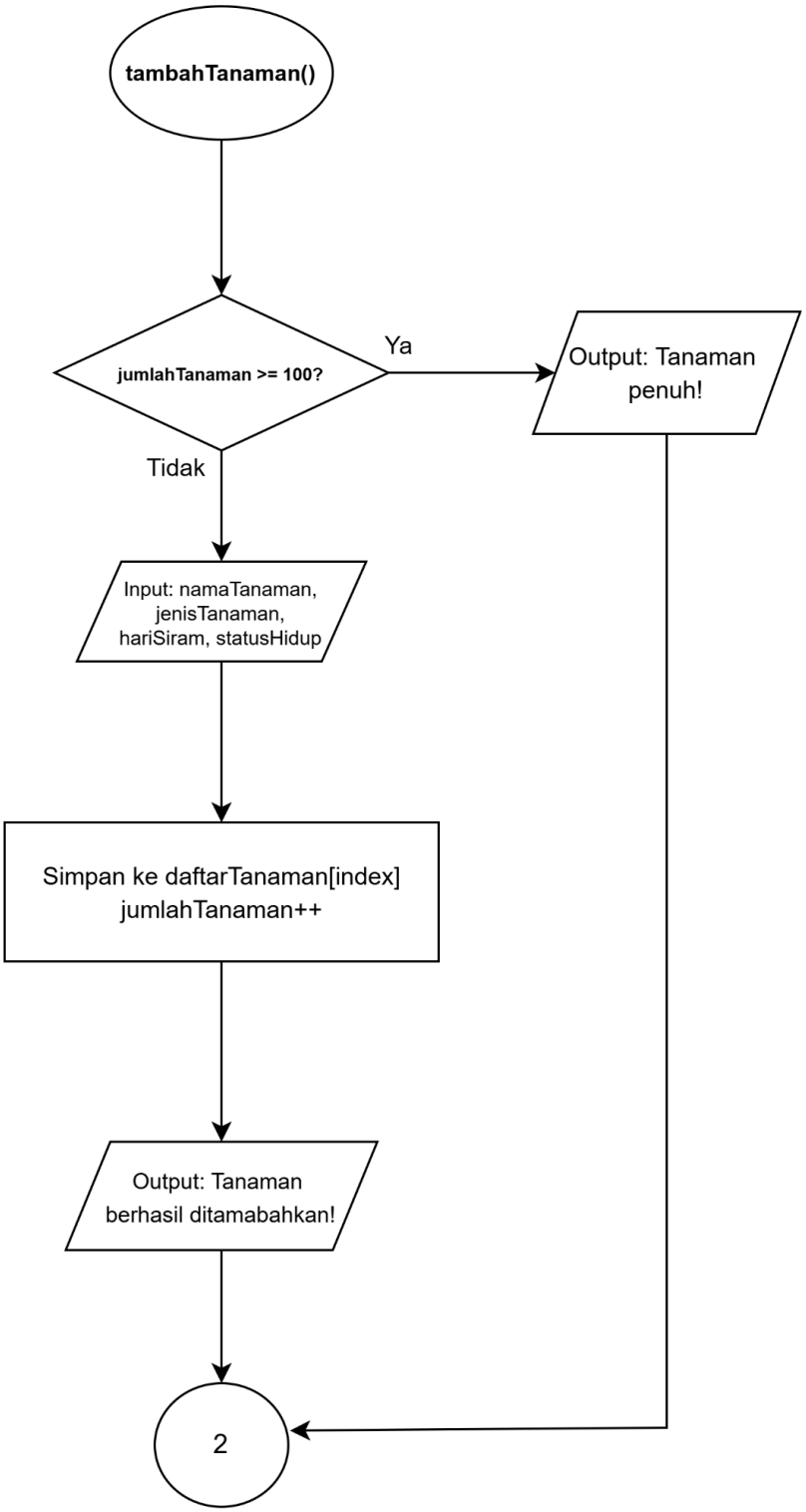
Gambar 1.1


loginUser(dataUser[],
jumlahUser, nama, nim)
i = 0
i < jumlahUser?
Ya
statusAktif == true &&
namaUser == &&
nimUser == nim ?
Ya
Tidak
Tidak
return - 1
(user tidak ditemukan)
return i
(index user ditemukan)
Return - 1
Return i
i++

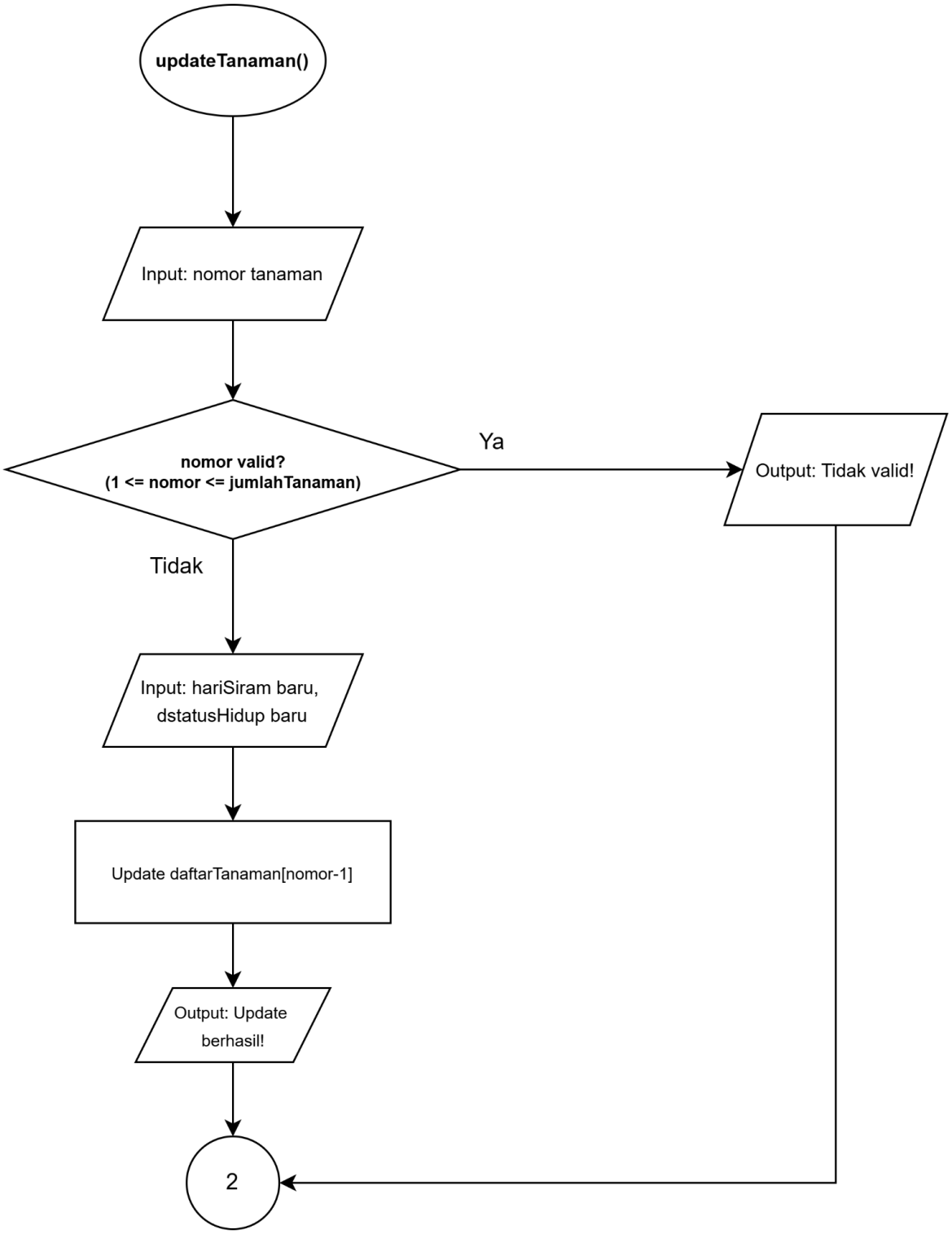
Gambar 1.2


registerUser()
jumlahUser >= 50 ?
Ya
Output: User penuh!
Tidak
Input: namaUser, nimUser
jumlahTanaman=0
statusAktif=true
jumlahUser++
Output: Register berhasil!
1

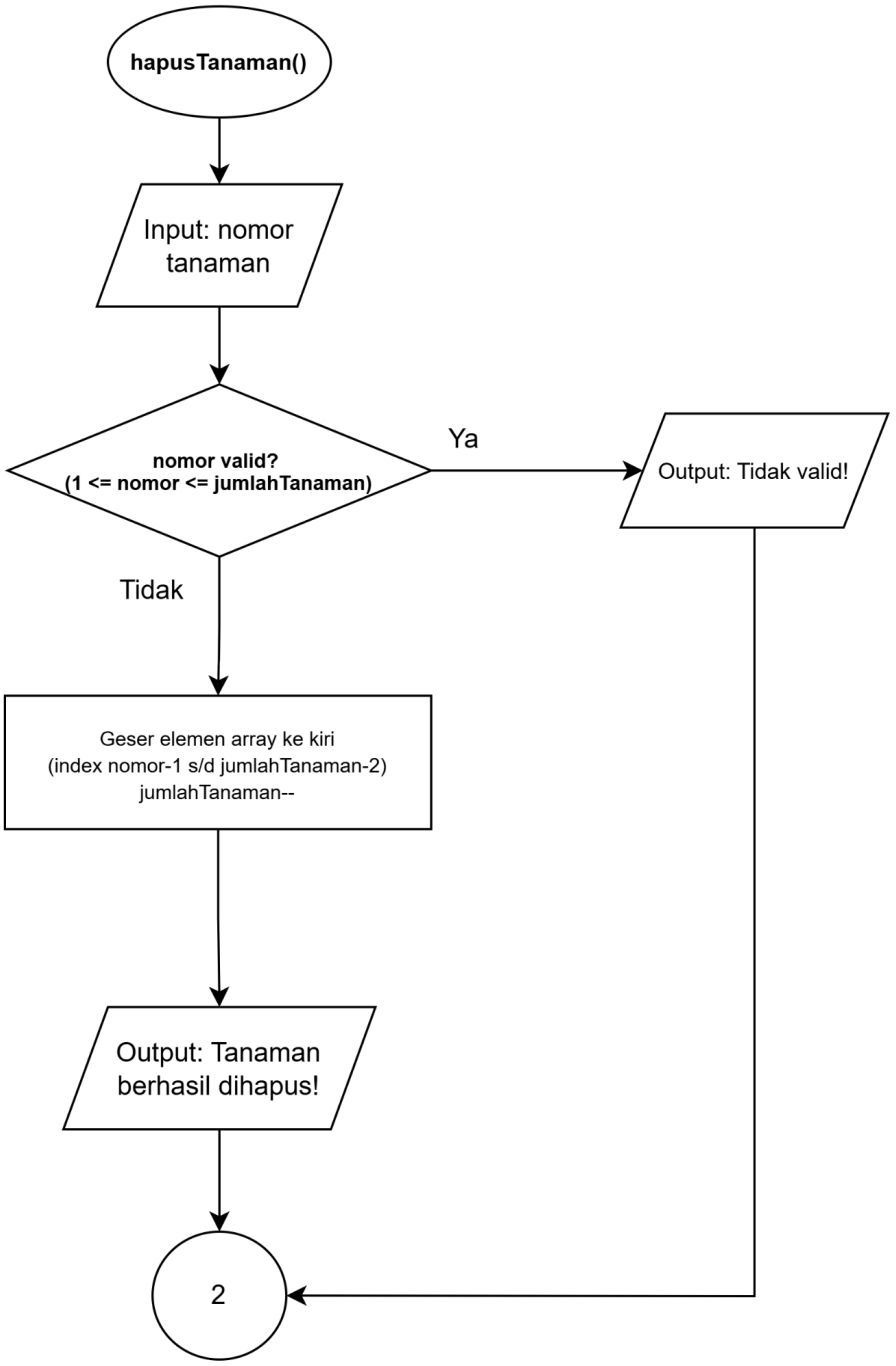
Gambar 1.3


tambahTanaman()
jumlahTanaman >= 100?
Ya
Output: Tanaman penuh!
Tidak
Input: namaTanaman, jenisTanaman, hariSiram, statusHidup
Simpan ke daftarTanaman[index]
jumlahTanaman++
Output: Tanaman berhasil ditamabahkan!
2

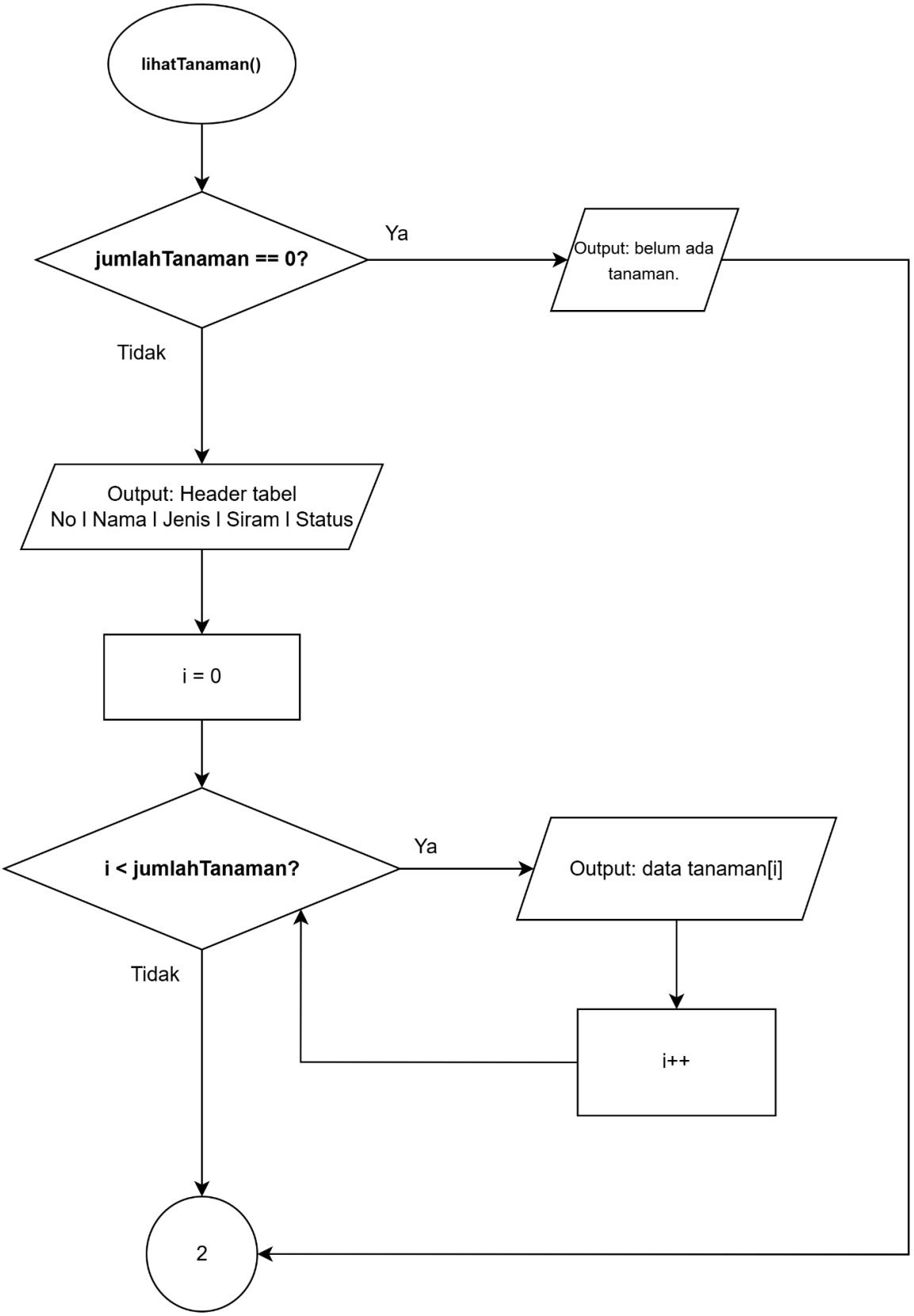
Gambar 1.4


updateTanaman()
Input: nomor tanaman
nomor valid?
(1 <= nomor <= jumlahTanaman)
Ya
Output: Tidak valid!
Tidak
Input: hariSiram baru,
dstatusHidup baru
Update daftarTanaman[nomor-1]
Output: Update
berhasil!
2

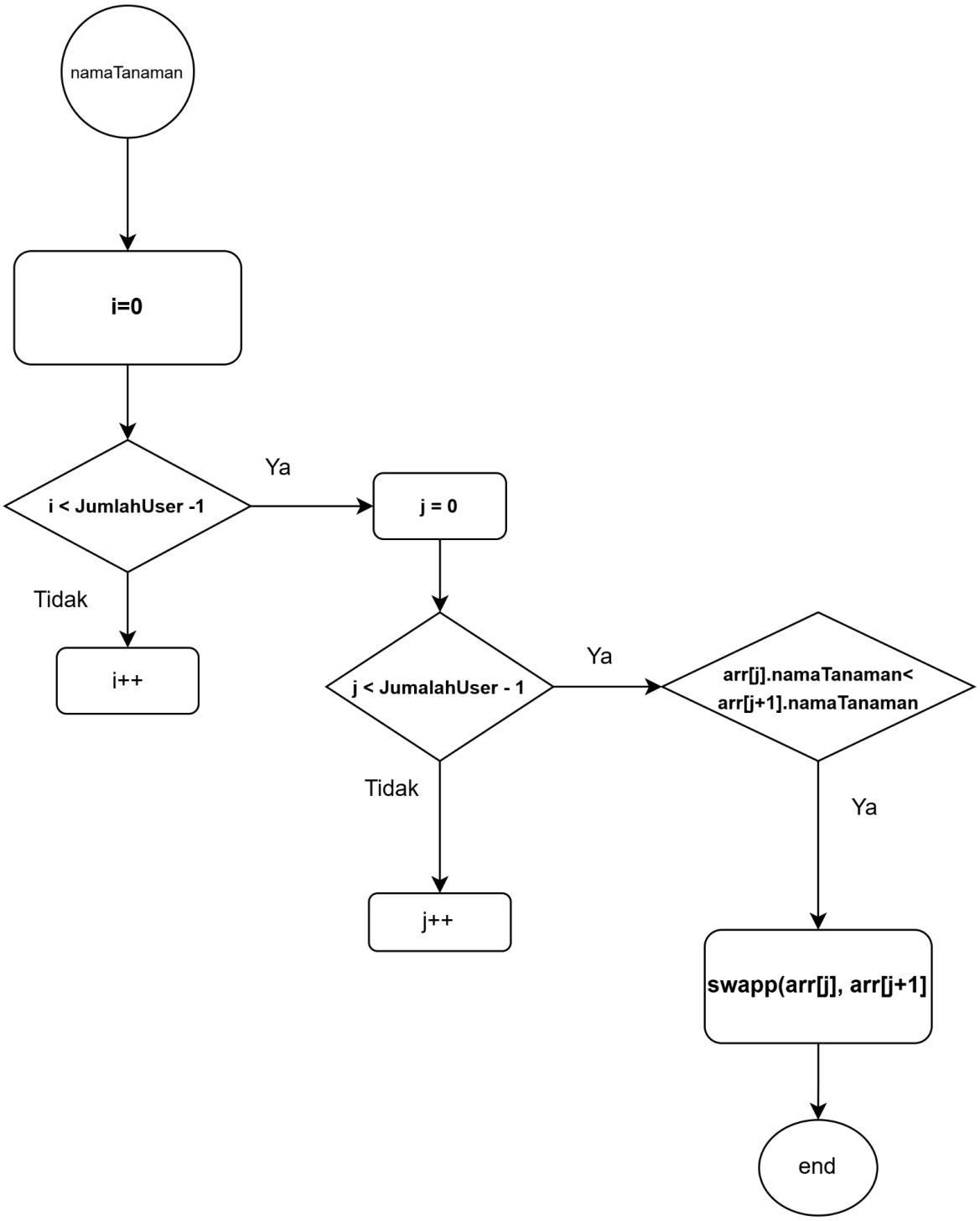
Gambar 1.5


hapusTanaman()
Input: nomor tanaman
nomor valid?
(1 <= nomor <= jumlahTanaman)
Ya
Output: Tidak valid!
Tidak
Geser elemen array ke kiri
(index nomor-1 s/d jumlahTanaman-2)
jumlahTanaman--
Output: Tanaman berhasil dihapus!
2

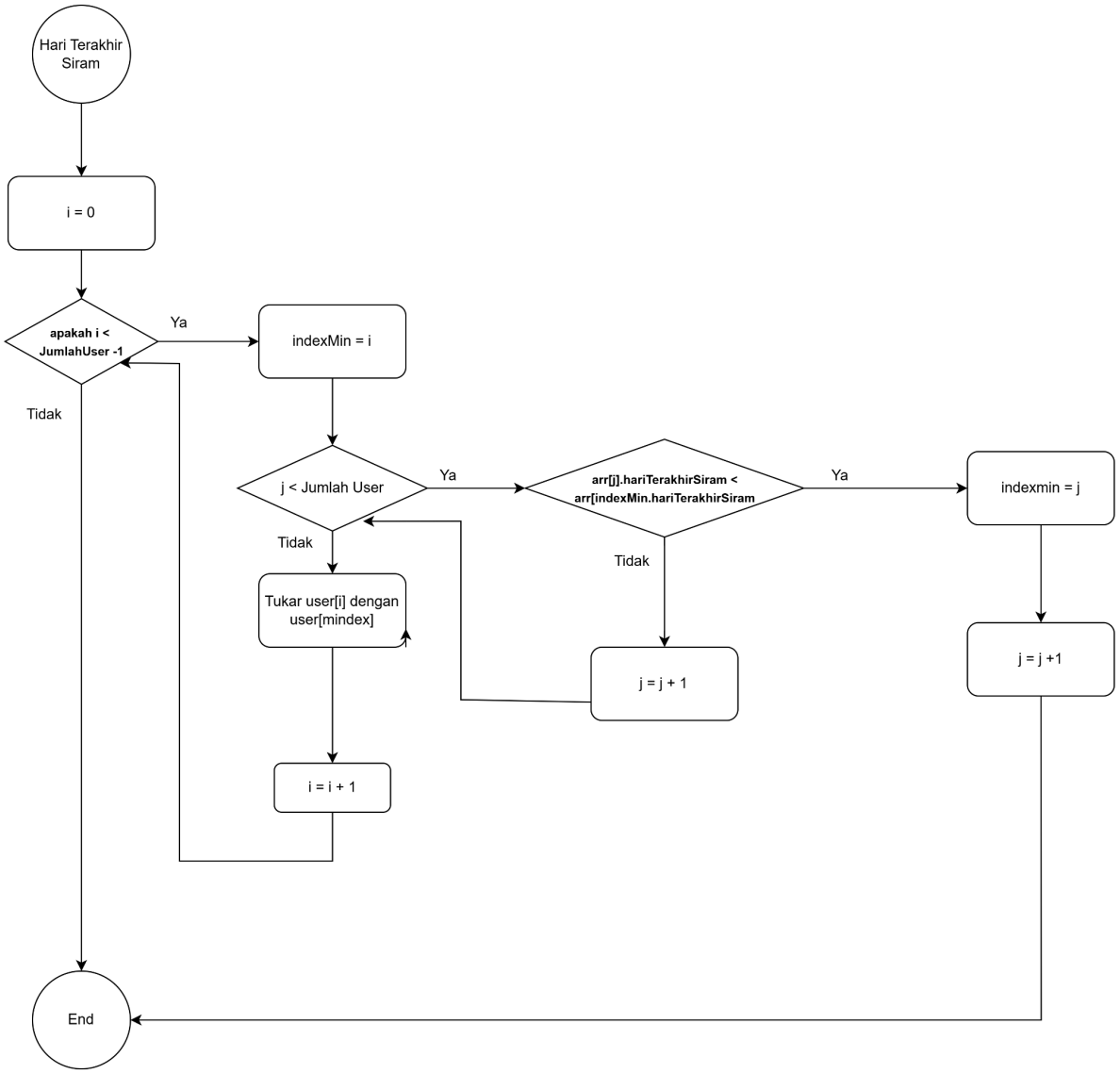
Gambar 1.6


lihatTanaman()
jumlahTanaman == 0?
Ya
Output: belum ada tanaman.
Tidak
Output: Header tabel
No I Nama I Jenis I Siram I Status
i = 0
i < jumlahTanaman?
Ya
Output: data tanaman[i]
Tidak
i++
2

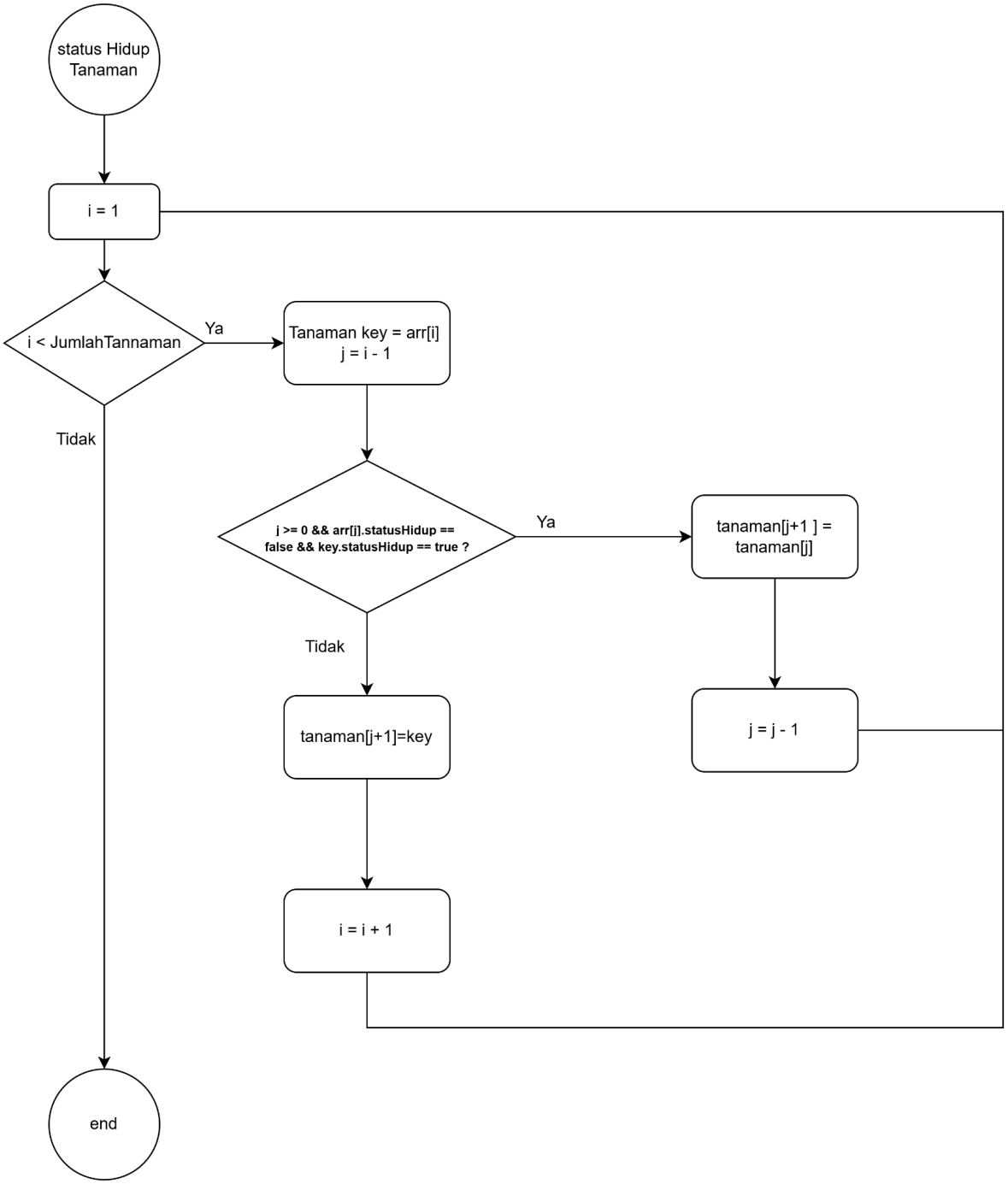
Gambar 1.7


namaTanaman
i=0
i < JumlahUser -1
Ya
j = 0
Tidak
i++
j < JumalahUser - 1
Ya
arr[j].namaTanaman<
arr[j+1].namaTanaman
Tidak
Ya
j++
swapp(arr[j], arr[j+1]
end

Gambar 1.8


Hari Terakhir Siram
i = 0
apakah i < JumlahUser -1
Ya
Tidak
indexMin = i
j < Jumlah User
Ya
Tidak
arr[j].hariTerakhirSiram < arr[indexMin.hariTerakhirSiram
Ya
Tidak
indexmin = j
Tukar user[i] dengan user[mindex]
j = j + 1
j = j +1
i = i + 1
End

Gambar 1.9


status Hidup Tanaman
i = 1
i < JumlahTannaman
Ya
Tidak
Tanaman key = arr[i]
j = i - 1
j >= 0 && arr[j].statusHidup == false && key.statusHidup == true ?
Ya
Tidak
tanaman[j+1] = tanaman[j]
j = j - 1
tanaman[j+1]=key
i = i + 1
end

Gambar 1.10

B. Penjelasan Singkat

Sistem Manajemen Tanaman Pribadi merupakan aplikasi berbasis konsol C++ yang mengelola data tanaman menggunakan konsep struct, pointer, dan alokasi memori dinamis (new/delete[]). Program menyediakan fitur autentikasi pengguna, operasi CRUD, tiga metode pengurutan (Bubble, Selection, Insertion), serta pencarian Linear dan Binary Search. Dalam pengembangannya, program telah ditingkatkan dengan menerapkan *Error Handling* berbasis *Exception Handling* (try, throw, catch) untuk mencegah program berhenti secara mendadak akibat kesalahan input, kegagalan alokasi memori, atau akses pointer null. Selain itu, kode program telah dimodularisasi menggunakan *User-Defined Library* (tanaman_lib.h) yang memisahkan deklarasi struktur data, kelas exception kustom, dan prototype fungsi. Pendekatan ini meningkatkan keterbacaan, reusabilitas, serta keamanan program. Validasi input juga diperketat melalui pengecekan tipe data (cin.fail()) dan batasan nilai, disertai dengan antarmuka yang lebih informatif dan terstruktur.

C. Source Code

a. TANAMAN_LIB.H

```
#ifndef TANAMAN_LIB_H
#define TANAMAN_LIB_H

#include <iostream>
#include <string>
#include <stdexcept>
#include <new>

using namespace std;

// Custom Exception Classes
class InvalidInputException : public invalid_argument {
public:
    InvalidInputException(const string& msg) : invalid_argument(msg) {}
};

class OutOfRangeException : public out_of_range {
public:
    OutOfRangeException(const string& msg) : out_of_range(msg) {}
};

class MemoryException : public bad_alloc {
public:
    MemoryException(const string& msg) : bad_alloc() { message = msg; }
    const char* what() const noexcept override { return message.c_str(); }
private:
    string message;
};

// Struct Definitions
struct Tanaman {
```

```
    string namaTanaman;
    string jenisTanaman;
    int hariTerakhirSiram;
    bool statusHidup;
};

struct User {
    string namaUser;
    string nimUser;
    Tanaman* daftarTanaman;
    int jumlahTanaman;
    bool statusAktif;
};

// Function Declarations
void sortNamaBubbleDesc(Tanaman arr[], int n);
void sortHariSelectionAsc(Tanaman arr[], int n);
void sortStatusInsertion(Tanaman arr[], int n);
int linearSearchNama(Tanaman* arr, int n, string target);
int binarySearchHari(Tanaman* arr, int n, int target);

int loginUser(User* dataUser, int jumlahUser, string nama, string nim);
void registerUser(User* dataUser, int &jumlahUser);
void tambahTanaman(User* user);
void lihatTanaman(User* user);
void updateTanaman(User* user);
void hapusTanaman(User* user);
void cleanupMemory(User* dataUser, int jumlahUser);

#endif
```

b. MAIN

```
#include <iostream>
#include <string>
#include <algorithm>
#include "TANAMAN_LIB.H"
using namespace std;

// ========== IMPLEMENTASI FUNGSI ==========

void sortNamaBubbleDesc(Tanaman arr[], int n) {
    bool swapped;
    for (int i = 0; i < n - 1; i++) {
        swapped = false;
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j].namaTanaman < arr[j + 1].namaTanaman) {
                swap(arr[j], arr[j + 1]);
```

```
                swapped = true;
            }
        }
        if (!swapped) break;
    }
}

void sortHariSelectionAsc(Tanaman arr[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int indeksMin = i;
        for (int j = i + 1; j < n; j++) {
            if (arr[j].hariTerakhirSiram < arr[indeksMin].hariTerakhirSiram)
{
                indeksMin = j;
            }
        }
        if (indeksMin != i) {
            swap(arr[i], arr[indeksMin]);
        }
    }
}

void sortStatusInsertion(Tanaman arr[], int n) {
    for (int i = 1; i < n; i++) {
        Tanaman key = arr[i];
        int j = i - 1;
        while (j >= 0 && arr[j].statusHidup == false && key.statusHidup ==
true) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
}

int linearSearchNama(Tanaman* arr, int n, string target) {
    for (int i = 0; i < n; i++) {
        if ((arr + i)->namaTanaman == target) {
            return i;
        }
    }
    return -1;
}

int binarySearchHari(Tanaman* arr, int n, int target) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
```

```
        if ((arr + mid)->hariTerakhirSiram == target) {
            return mid;
        } else if ((arr + mid)->hariTerakhirSiram < target) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return -1;
}

int loginUser(User* dataUser, int jumlahUser, string nama, string nim) {
    for (int i = 0; i < jumlahUser; i++) {
        if ((dataUser + i)->statusAktif &&
            (dataUser + i)->namaUser == nama &&
            (dataUser + i)->nimUser == nim) {
            return i;
        }
    }
    return -1;
}

void registerUser(User* dataUser, int &jumlahUser) {
    try {
        if (jumlahUser >= 50) {
            throw OutOfRangeException("User penuh! Maksimal 50 user.");
        }

        cin.ignore();
        cout << "Nama: ";
        string nama;
        getline(cin, nama);
        if (nama.empty()) {
            throw InvalidInputException("Nama tidak boleh kosong!");
        }

        cout << "NIM: ";
        string nim;
        getline(cin, nim);
        if (nim.empty()) {
            throw InvalidInputException("NIM tidak boleh kosong!");
        }

        (dataUser + jumlahUser)->daftarTanaman = new Tanaman[100];
        (dataUser + jumlahUser)->namaUser = nama;
        (dataUser + jumlahUser)->nimUser = nim;
        (dataUser + jumlahUser)->jumlahTanaman = 0;
        (dataUser + jumlahUser)->statusAktif = true;
```

```
        jumlahUser++;

        cout << " Register berhasil!" << endl;

    } catch (const bad_alloc&) {
        throw MemoryException("Gagal alokasi memori untuk user baru!");
    }
}

void tambahTanaman(User* user) {
    try {
        if (user == nullptr) {
            throw InvalidInputException("User tidak valid!");
        }

        if (user->jumlahTanaman >= 100) {
            throw OutOfRangeException("Kapasitas tanaman penuh (maks
100)!");
        }

        int index = user->jumlahTanaman;
        cin.ignore();

        cout << "Nama Tanaman: ";
        string nama;
        getline(cin, nama);
        if (nama.empty()) {
            throw InvalidInputException("Nama tanaman tidak boleh kosong!");
        }
        (user->daftarTanaman + index)->namaTanaman = nama;

        cout << "Jenis: ";
        string jenis;
        getline(cin, jenis);
        if (jenis.empty()) {
            throw InvalidInputException("Jenis tanaman tidak boleh
kosong!");
        }
        (user->daftarTanaman + index)->jenisTanaman = jenis;

        cout << "Hari Siram (1-7): ";
        int hari;
        cin >> hari;
        if (cin.fail()) {
            cin.clear();
            cin.ignore(1000, '\n');
            throw InvalidInputException("Input harus berupa angka!");
        }
```

```
        if (hari < 1 || hari > 7) {
            throw OutOfRangeException("Hari siram harus antara 1-7!");
        }
        (user->daftarTanaman + index)->hariTerakhirSiram = hari;

        cout << "Status Hidup (1=Ya, 0=Tidak): ";
        int status;
        cin >> status;
        if (cin.fail()) {
            cin.clear();
            cin.ignore(1000, '\n');
            throw InvalidInputException("Input harus berupa angka!");
        }
        if (status != 0 && status != 1) {
            throw OutOfRangeException("Status hanya boleh 0 atau 1!");
        }
        (user->daftarTanaman + index)->statusHidup = (status == 1);

        user->jumlahTanaman++;
        cout << " Tanaman berhasil ditambahkan!" << endl;

    } catch (const bad_alloc&) {
        cout << " Memory Error: Gagal alokasi memori!" << endl;
    } catch (const exception& e) {
        cout << " Error: " << e.what() << endl;
    }
}

void lihatTanaman(User* user) {
    try {
        if (user == nullptr) {
            throw InvalidInputException("User tidak valid!");
        }

        if (user->jumlahTanaman == 0) {
            cout << " Belum ada tanaman." << endl;
            return;
        }

    cout << "\nNo | Nama           | Jenis          | Siram | Status" <<
endl;
    cout << "-------------------------------------------------------" <<
endl;

        for (int i = 0; i < user->jumlahTanaman; i++) {
            cout << "| " << (i + 1);
            cout.width(2); cout << "| ";
            cout.width(14); cout << left << (user->daftarTanaman +
```

```
i)->namaTanaman << " | ";
            cout.width(14); cout << left << (user->daftarTanaman +
i)->jenisTanaman << " | ";
            cout.width(5); cout << right << (user->daftarTanaman +
i)->hariTerakhirSiram << " | ";
            cout << ((user->daftarTanaman + i)->statusHidup ? "Hidup  |" :
"Mati   |") << endl;
        }
        cout << "=====================================================" <<
endl;

    } catch (const exception& e) {
        cout << " Error: " << e.what() << endl;
    }
}

void updateTanaman(User* user) {
    try {
        if (user == nullptr) {
            throw InvalidInputException("User tidak valid!");
        }

        if (user->jumlahTanaman == 0) {
            cout << " Belum ada tanaman untuk diupdate!" << endl;
            return;
        }

        int nomor;
        cout << "Nomor tanaman: ";
        cin >> nomor;

        if (cin.fail()) {
            cin.clear();
            cin.ignore(1000, '\n');
            throw InvalidInputException("Input harus berupa angka!");
        }

        if (nomor <= 0 || nomor > user->jumlahTanaman) {
            throw OutOfRangeException("Nomor tidak valid! Pilih 1-" +
to_string(user->jumlahTanaman));
        }

        int index = nomor - 1;

        cout << "Hari Siram Baru (1-7): ";
        int hari;
        cin >> hari;
        if (cin.fail()) {
```

```
            cin.clear();
            cin.ignore(1000, '\n');
            throw InvalidInputException("Input harus berupa angka!");
        }
        if (hari < 1 || hari > 7) {
            throw OutOfRangeException("Hari siram harus antara 1-7!");
        }
        (user->daftarTanaman + index)->hariTerakhirSiram = hari;

        cout << "Status Hidup Baru (1=Ya, 0=Tidak): ";
        int status;
        cin >> status;
        if (cin.fail()) {
            cin.clear();
            cin.ignore(1000, '\n');
            throw InvalidInputException("Input harus berupa angka!");
        }
        if (status != 0 && status != 1) {
            throw OutOfRangeException("Status hanya boleh 0 atau 1!");
        }
        (user->daftarTanaman + index)->statusHidup = (status == 1);

        cout << " Update berhasil!" << endl;

    } catch (const exception& e) {
        cout << " Error: " << e.what() << endl;
    }
}

void hapusTanaman(User* user) {
    try {
        if (user == nullptr) {
            throw InvalidInputException("User tidak valid!");
        }

        if (user->jumlahTanaman == 0) {
            cout << " Belum ada tanaman untuk dihapus!" << endl;
            return;
        }

        int nomor;
        cout << "Nomor tanaman: ";
        cin >> nomor;

        if (cin.fail()) {
            cin.clear();
            cin.ignore(1000, '\n');
            throw InvalidInputException("Input harus berupa angka!");
```

```
        }

        if (nomor <= 0 || nomor > user->jumlahTanaman) {
            throw OutOfRangeException("Nomor tidak valid! Pilih 1-" +
to_string(user->jumlahTanaman));
        }

        for (int i = nomor - 1; i < user->jumlahTanaman - 1; i++) {
            *(user->daftarTanaman + i) = *(user->daftarTanaman + i + 1);
        }
        user->jumlahTanaman--;
        cout << " Tanaman berhasil dihapus!" << endl;

    } catch (const exception& e) {
        cout << " Error: " << e.what() << endl;
    }
}

void cleanupMemory(User* dataUser, int jumlahUser) {
    for (int i = 0; i < jumlahUser; i++) {
        if ((dataUser + i)->daftarTanaman != nullptr) {
            delete[] (dataUser + i)->daftarTanaman;
        }
    }
}

// ================= MAIN =================
int main() {
    try {
        User* dataUser = new User[50];
        int jumlahUser = 0;
        int indexUserAktif = -1;
        bool sudahLogin = false;
        bool program = true;

        for (int i = 0; i < 50; i++) {
            (dataUser + i)->daftarTanaman = nullptr;
            (dataUser + i)->statusAktif = false;
        }


        cout << "===== SISTEM PENGELOLAAN TANAMAN =====" << endl;

        while (program) {
            try {
                if (!sudahLogin) {
                    cout << "=====LOGIN MENU=====" << endl;
                    cout << " 1. Login              " << endl;
```

```
                    cout << " 2. Register           " << endl;
                    cout << " 3. Keluar             " << endl;

                    cout << "Pilihan: ";

                    int pilih;
                    cin >> pilih;

                    if (cin.fail()) {
                        cin.clear();
                        cin.ignore(1000, '\n');
                        throw InvalidInputException("Pilihan harus angka!");
                    }

                    if (pilih < 1 || pilih > 3) {
                        throw OutOfRangeException("Pilihan hanya 1-3!");
                    }

                    if (pilih == 1) {
                        string nama, nim;
                        cin.ignore();
                        cout << "Nama: "; getline(cin, nama);
                        cout << "NIM: "; getline(cin, nim);
                        indexUserAktif = loginUser(dataUser, jumlahUser,
nama, nim);
                        if (indexUserAktif != -1) {
                            sudahLogin = true;
                            cout << " Login berhasil! Selamat datang, " <<
nama << endl;
                        } else {
                            cout << " Login gagal! User tidak ditemukan." <<
endl;
                        }
                    } else if (pilih == 2) {
                        registerUser(dataUser, jumlahUser);
                    } else if (pilih == 3) {
                        program = false;
                        cout << " Terima kasih telah menggunakan program
ini!" << endl;
                    }

                } else {

                    cout << "=====MENU TANAMAN=====" << endl;
                    cout << "  1. Tambah Tanaman          " << endl;
                    cout << "  2. Lihat Tanaman           " << endl;
                    cout << "  3. Update Tanaman          " << endl;
                    cout << "  4. Hapus Tanaman           " << endl;
```

```
                    cout << "  5. Logout                    " << endl;
                    cout << "  6. Urutkan Tanaman           " << endl;
                    cout << "  7. Cari Tanaman              " << endl;
                    cout << "Pilihan: ";

                    int pilih;
                    cin >> pilih;

                    if (cin.fail()) {
                        cin.clear();
                        cin.ignore(1000, '\n');
                        throw InvalidInputException("Pilihan harus angka!");
                    }

                    if (pilih < 1 || pilih > 7) {
                        throw OutOfRangeException("Pilihan hanya 1-7!");
                    }

                    if (pilih == 1)
tambahTanaman(&dataUser[indexUserAktif]);
                    else if (pilih == 2)
lihatTanaman(&dataUser[indexUserAktif]);
                    else if (pilih == 3)
updateTanaman(&dataUser[indexUserAktif]);
                    else if (pilih == 4)
hapusTanaman(&dataUser[indexUserAktif]);
                    else if (pilih == 5) {
                        sudahLogin = false;
                        indexUserAktif = -1;
                        cout << " Logout berhasil." << endl;
                    }
                    else if (pilih == 6) {
                        cout << "=====URUTKAN TANAMAN=====" << endl;
                        cout << " 1. Nama Tanaman (Z-A) [Bubble Sort]   " <<
endl;
                        cout << " 2. Hari Siram (Kecil-Besar) [Select]  " <<
endl;
                        cout << " 3. Status Hidup (Hidup dulu) [Insert] " <<
endl;
                        cout << "Pilihan: ";

                        int subPilih;
                        cin >> subPilih;

                        if (cin.fail()) {
                            cin.clear();
                            cin.ignore(1000, '\n');
                            throw InvalidInputException("Pilihan harus
```

```
angka!");
                        }

                        if (subPilih < 1 || subPilih > 3) {
                            throw OutOfRangeException("Pilihan hanya 1-3!");
                        }

                        Tanaman* arr =
dataUser[indexUserAktif].daftarTanaman;
                        int count = dataUser[indexUserAktif].jumlahTanaman;

                        if (count < 2) {
                            cout << "  Data kurang dari 2, tidak perlu
sorting." << endl;
                        } else {
                            if (subPilih == 1) {
                                sortNamaBubbleDesc(arr, count);
                                cout << " Diurutkan (Nama Descending)!" <<
endl;
                            }
                            else if (subPilih == 2) {
                                sortHariSelectionAsc(arr, count);
                                cout << " Diurutkan (Hari Siram Ascending)!"
<< endl;
                            }
                            else if (subPilih == 3) {
                                sortStatusInsertion(arr, count);
                                cout << " Diurutkan (Status Hidup)!" <<
endl;
                            }
                            lihatTanaman(&dataUser[indexUserAktif]);
                        }
                    }
                    // ================= FITUR CARI (SEARCHING)
=================
                    else if (pilih == 7) {
                        cout << "=====CARI TANAMAN=====" << endl;
                        cout << " 1. Cari Nama              " << endl;
                        cout << " 2. Cari Hari Siram        " << endl;
                        cout << "Pilihan: ";

                        int cariPilih;
                        cin >> cariPilih;

                        if (cin.fail()) {
                            cin.clear();
                            cin.ignore(1000, '\n');
                            throw InvalidInputException("Pilihan harus
```

```
angka!");
                        }

                        if (cariPilih < 1 || cariPilih > 2) {
                            throw OutOfRangeException("Pilihan hanya 1-2!");
                        }

                        Tanaman* arr =
dataUser[indexUserAktif].daftarTanaman;
                        int count = dataUser[indexUserAktif].jumlahTanaman;

                        if (count == 0) {
                            cout << " Belum ada data tanaman." << endl;
                        } else {
                            if (cariPilih == 1) {
                                cin.ignore();
                                string target;
                                cout << "Masukkan nama tanaman: ";
                                getline(cin, target);

                                if (target.empty()) {
                                    throw InvalidInputException("Nama tidak
boleh kosong!");
                                }

                                int hasil = linearSearchNama(arr, count,
target);
                                if (hasil != -1) {
                                    cout << " Ditemukan di No. " << hasil +
1 << endl;
                                    cout << "Nama: " << (arr +
hasil)->namaTanaman
                                         << " | Status: " << ((arr +
hasil)->statusHidup ? "Hidup" : "Mati") << endl;
                                } else {
                                    cout << " Tanaman '" << target << "'
tidak ditemukan." << endl;
                                }
                            } else if (cariPilih == 2) {
                                int target;
                                cout << "Masukkan hari terakhir siram (1-7):
";
                                cin >> target;

                                if (cin.fail()) {
                                    cin.clear();
                                    cin.ignore(1000, '\n');
                                    throw InvalidInputException("Input harus
```

```
angka!");
                                }

                                if (target < 1 || target > 7) {
                                    throw OutOfRangeException("Hari harus
1-7!");
                                }

                                sortHariSelectionAsc(arr, count);
                                cout << "(Data diurutkan otomatis untuk
Binary Search)" << endl;

                                int hasil = binarySearchHari(arr, count,
target);
                                if (hasil != -1) {
                                    cout << " Ditemukan di No. " << hasil +
1 << endl;
                                    cout << "Nama: " << (arr +
hasil)->namaTanaman
                                         << " | Hari: " << (arr +
hasil)->hariTerakhirSiram << endl;
                                } else {
                                    cout << " Tidak ada tanaman dengan hari
siram '" << target << "'." << endl;
                                }
                            }
                        }
                    }
                }
            } catch (const InvalidInputException& e) {
                cout << " INPUT ERROR: " << e.what() << endl;
                cout << " Silakan coba lagi dengan input yang valid." <<
endl;
            } catch (const OutOfRangeException& e) {
                cout << " RANGE ERROR: " << e.what() << endl;
                cout << " Silakan pilih nilai yang sesuai." << endl;
            } catch (const exception& e) {
                cout << " UNEXPECTED ERROR: " << e.what() << endl;
                cout << " Program akan melanjutkan..." << endl;
            }
        }

        cleanupMemory(dataUser, jumlahUser);
        delete[] dataUser;
        cout << " Program selesai. Memory telah dibersihkan." << endl;

    } catch (const bad_alloc&) {
        cerr << " FATAL ERROR: Gagal alokasi memori!" << endl;
```

```
        return 1;
    } catch (const exception& e) {
        cerr << " FATAL ERROR: " << e.what() << endl;
        return 1;
    }

    return 0;
}
```

D. Output

```
=== LOGIN MENU ===
1. Login
2. Register
3. Keluar
```

Gambar 3.1 *Output Menu Login*

```
=== LOGIN MENU ===
1. Login
2. Register
3. Keluar
Pilihan: 2
Nama: Atma
NIM: 023
Register berhasil!
```

Gambar 3.2 *Output Menu Register*

```
=== LOGIN MENU ===
1. Login
2. Register
3. Keluar
Pilihan: 1
Nama: Atma
NIM: 023
Login berhasil!
```

Gambar 3.3 *Output Menu Login*

```
=== MENU TANAMAN ===
1. Tambah
2. Lihat
3. Update
4. Hapus
5. Logout
6. Urutkan
Pilihan:
```

Gambar 3.4 *Output Menu Tanaman*

```
=== MENU TANAMAN ===
1. Tambah
2. Lihat
3. Update
4. Hapus
5. Logout
6. Urutkan
Pilihan: 1
Nama Tanaman: anggrek
Jenis: bunga
Hari Siram (1-7): 3
Status Hidup (1=Ya, 0=Tidak): 1
Tanaman berhasil ditambahkan!
```

Gambar 3.5 *Output Menu Tambah Tanaman*

```
=== MENU TANAMAN ===
1. Tambah
2. Lihat
3. Update
4. Hapus
5. Logout
6. Urutkan
Pilihan: 2

No | Nama           | Jenis          | Siram | Status
-------------------------------------------------------
1  | anggrek | bunga | 3     | Hidup
2  | kamboja | bunga | 5     | Hidup
3  | kol | sayur | 6     | Mati
4  | sawi | sayur | 1     | Mati
5  | apel | buah | 1     | Mati
6  | kacang tanah | kacang | 4     | Hidup
-------------------------------------------------------
```

Gambar 3.5 *Output Menu Lihat Tanaman*

```
=== MENU TANAMAN ===
1. Tambah
2. Lihat
3. Update
4. Hapus
5. Logout
6. Urutkan
Pilihan: 3
Nomor tanaman: 3
Hari Siram Baru: 7
Status Hidup Baru (1=Ya, 0=Tidak): 1
Update berhasil!
```

Gambar 3.6 *Output Menu Update Tanaman*

```
No | Nama          | Jenis         | Siram | Status
-----------------------------------------------------
1  | anggrek | bunga | 3     | Hidup
2  | kamboja | bunga | 5     | Hidup
3  | kol | sayur | 7     | Hidup
4  | sawi | sayur | 1     | Mati
5  | apel | buah | 1     | Mati
6  | kacang tanah | kacang | 4     | Hidup
-----------------------------------------------------
```

Gambar 3.7 *Output Daftar Tanaman setelah di Update*

```
=== MENU TANAMAN ===
1. Tambah
2. Lihat
3. Update
4. Hapus
5. Logout
6. Urutkan
Pilihan: 4
Nomor tanaman: 6
Tanaman berhasil dihapus!
```

Gambar 3.8 *Output Menu Hapus Tanaman*

```
=== MENU TANAMAN ===
1. Tambah
2. Lihat
3. Update
4. Hapus
5. Logout
6. Urutkan
Pilihan: 2

No | Nama          | Jenis         | Siram | Status
-----------------------------------------------------
1  | anggrek | bunga | 3     | Hidup
2  | kamboja | bunga | 5     | Hidup
3  | kol | sayur | 7     | Hidup
4  | sawi | sayur | 1     | Mati
5  | apel | buah | 1     | Mati
-----------------------------------------------------
```

Gambar 3.9 *Output Daftar Tanaman setelah di Hapus*

```
=== MENU TANAMAN ===
1. Tambah
2. Lihat
3. Update
4. Hapus
5. Logout
6. Urutkan
Pilihan: 6

Pilih metode sorting:
1. Nama Tanaman (Huruf Descending: Z s/d A) [Bubble Sort]
2. Hari Terakhir Siram (Angka Ascending: Kecil s/d Besar) [Selection Sort]
3. Status Hidup (Hidup diutamakan s/d Mati) [Insertion Sort]
Pilihan:
```

Gambar 3.10 *Output Menu Urutkan*

```
Pilih metode sorting:
1. Nama Tanaman (Huruf Descending: Z s/d A) [Bubble Sort]
2. Hari Terakhir Siram (Angka Ascending: Kecil s/d Besar) [Selection Sort]
3. Status Hidup (Hidup diutamakan s/d Mati) [Insertion Sort]
Pilihan: 1
Diurutkan dengan Bubble Sort (Nama Descending)!

No | Nama          | Jenis         | Siram | Status
-------------------------------------------------------
1  | sawi | sayur | 1     | Mati
2  | kol | sayur | 7     | Hidup
3  | kamboja | bunga | 5     | Hidup
4  | apel | buah | 1     | Mati
5  | anggrek | bunga | 3     | Hidup
-------------------------------------------------------
```

Gambar 3.11 *Output Menu Urutkan Nama Z-A*

```
=== MENU TANAMAN ===
1. Tambah
2. Lihat
3. Update
4. Hapus
5. Logout
6. Urutkan
Pilihan: 6

Pilih metode sorting:
1. Nama Tanaman (Huruf Descending: Z s/d A) [Bubble Sort]
2. Hari Terakhir Siram (Angka Ascending: Kecil s/d Besar) [Selection Sort]
3. Status Hidup (Hidup diutamakan s/d Mati) [Insertion Sort]
Pilihan: 2
Diurutkan dengan Selection Sort (Hari Siram Ascending)!

No | Nama          | Jenis         | Siram | Status
-------------------------------------------------------
1  | sawi | sayur | 1     | Mati
2  | apel | buah | 1     | Mati
3  | anggrek | bunga | 3     | Hidup
4  | kamboja | bunga | 5     | Hidup
5  | kol | sayur | 7     | Hidup
-------------------------------------------------------
```

Gambar 3.12 *Output Menu Urutkan Hari Terakhir Siram*

```
=== MENU TANAMAN ===
1. Tambah
2. Lihat
3. Update
4. Hapus
5. Logout
6. Urutkan
Pilihan: 6

Pilih metode sorting:
1. Nama Tanaman (Huruf Descending: Z s/d A) [Bubble Sort]
2. Hari Terakhir Siram (Angka Ascending: Kecil s/d Besar) [Selection Sort]
3. Status Hidup (Hidup diutamakan s/d Mati) [Insertion Sort]
Pilihan: 3
Diurutkan dengan Insertion Sort (Status Hidup)!

No | Nama           | Jenis          | Siram | Status
-------------------------------------------------------
1  | anggrek | bunga | 3     | Hidup
2  | kamboja | bunga | 5     | Hidup
3  | kol | sayur | 7     | Hidup
4  | sawi | sayur | 1     | Mati
5  | apel | buah | 1     | Mati
-------------------------------------------------------
```

Gambar 3.13 *Output Menu Urutkan Status Hidup*

```
=== MENU TANAMAN ===
1. Tambah
2. Lihat
3. Update
4. Hapus
5. Logout
6. Urutkan
7. Cari
Pilihan: 7
=== CARI TANAMAN ===
1. Cari Nama
2. Cari Hari Siram
Pilihan:
```

Gambar 3.14 *Output Menu Cari*

```
=== MENU TANAMAN ===
1. Tambah
2. Lihat
3. Update
4. Hapus
5. Logout
6. Urutkan
7. Cari
Pilihan: 7
=== CARI TANAMAN ===
1. Cari Nama
2. Cari Hari Siram
Pilihan: 1
Masukkan nama tanaman: apel
Ditemukan di No. 5
Nama: apel | Status: Mati
```

Gambar 3.15 *Output Menu Cari Nama Tanaman*

```
=== MENU TANAMAN ===
1. Tambah
2. Lihat
3. Update
4. Hapus
5. Logout
6. Urutkan
7. Cari
Pilihan: 7
=== CARI TANAMAN ===
1. Cari Nama
2. Cari Hari Siram
Pilihan: 2
Masukkan hari terakhir siram (1-7): 7
(Data diurutkan otomatis untuk Binary Search)
Ditemukan di No. 5
Nama: kol | Hari: 7
```

Gambar 3.16 *Output Menu Cari Hari Siram Tanaman*

E. Langkah-langkah GIT

Untuk menandai file yang ingin disimpan perubahannya.

```
PS C:\Praktikum-apl\posttest\post-test-apl-6> git add .
```

5.1 Git Add

Dapat menyimpan perubahan yang sudah kita add tadi ke dalam riwayat Git.

```
PS C:\Praktikum-apl\posttest\post-test-apl-6> git commit -m "tanaman1"
[main 4e6d55d] tanaman1
 2 files changed, 316 insertions(+)
 create mode 100644 posttest/post-test-apl-6/2509106086-KikaniaAgarista-PT-6.cpp
 create mode 100644 posttest/post-test-apl-6/2509106086-KikaniaAgarista-PT-6.exe
```

5.2 Git Commit

Mengirim commit dari repository lokal ke repository online (GitHub).

```
PS C:\Praktikum-apl\posttest\post-test-apl-6> git push -u origin main
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (6/6), 54.19 KiB | 9.03 MiB/s, done.
Total 6 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/kikaniaagarista-dot/Pratikum-apl.git
   f778a75..4e6d55d  main -> main
branch 'main' set up to track 'origin/main'.
```

5.3 Git Push